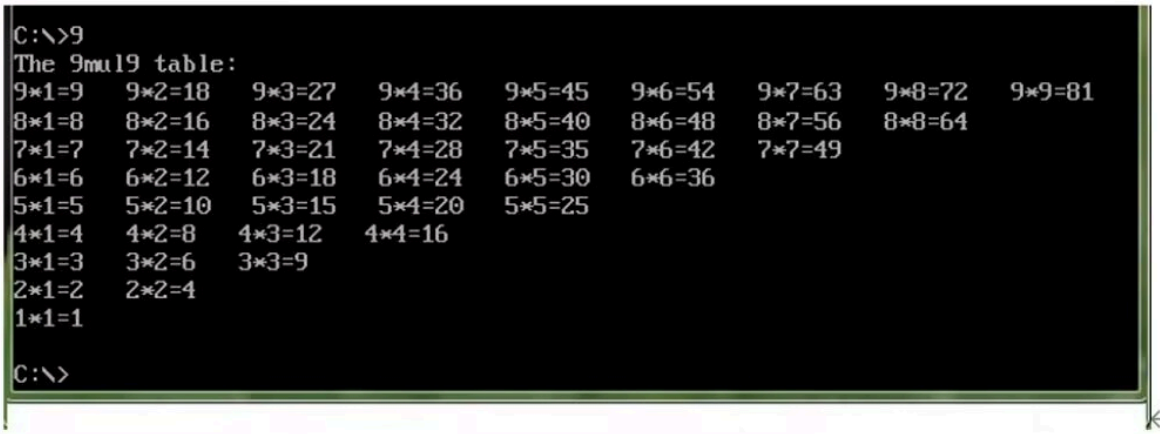
汇编语言第四次作业

作业题目 4：乘法指令和过程调用

一、输出九九乘法表

基本要求：至少实现一个过程调用，能正确调用并返回。（Call 和 ret 指令）

结果输出：



1）用 Call 和 ret 指令实现过程调用，过程（process）中需要用到主过程中寄存器资源（掌握寄存器在过程调用前后的保存和恢复）。

2）复习双循环的实现；

3）用 C 语言实现后察看反汇编代码并加注释；

第一步：明确要实现的效果

目标是输出倒三角形的九九乘法表，例如：

代码块

```
The 9 multiplication table:
9*1=9  9*2=18  9*3=27 ... 9*9=81
8*1=8  8*2=16  8*3=24 ... 8*8=64
...
2*1=2  2*2=4
1*1=1
```

这里有两个循环：

- 外层循环：控制被乘数，从 9 递减到 1。
- 内层循环：控制乘数，从 1 递增到当前被乘数。

例如外层循环取 `i = 5` 时，内层循环就依次计算 `5×1`、`5×2`、`5×3`、`5×4`、`5×5`。

此外，我们会设计一个专门负责输出乘法表达式的子程序，通过 `CALL` 调用、通过 `RET` 返回，满足作业要求。

第二步：编写九九乘法表汇编程序

在 `D:\myassembly` 目录下，新建一个文件，命名为 `multable.asm`。

代码块

```
.MODEL SMALL
.STACK 100H

.DATA
msg DB 'The 9 multiplication table:', 13, 10, '$'
row DB 9
col DB 1

.CODE
main PROC
    mov ax, @data
    mov ds, ax

    lea dx, msg
    mov ah, 09H
    int 21H

    mov row, 9

OuterLoop:
    cmp row, 0
    je Done

    mov col, 1

InnerLoop:
    mov al, col
    cmp al, row
    ja NextRow

    ; Output row
    mov dl, row
    add dl, '0'
    call PrintChar

    ; Output *
    mov dl, '*'
```

```
38      call PrintChar
39
40      ; Output col
41      mov dl, col
42      add dl, '0'
43      call PrintChar
44
45      ; Output =
46      mov dl, '='
47      call PrintChar
48
49      ; Calculate row * col
50      mov al, row
51      mul col
52      call PrintNumber
53
54      ; Output two spaces
55      mov dl, ' '
56      call PrintChar
57      call PrintChar
58
59      inc col
60      jmp InnerLoop
61
62  NextRow:
63      mov dl, 13
64      call PrintChar
65      mov dl, 10
66      call PrintChar
67
68      dec row
69      jmp OuterLoop
70
71  Done:
72      mov ax, 4C00H
73      int 21H
74  main ENDP
75
76  ; Output the character in DL
77  PrintChar PROC NEAR
78      push ax
79      push dx
80
81      mov ah, 02H
82      int 21H
83
84      pop dx
```

```
    pop ax
    ret
PrintChar ENDP

; Output the unsigned integer in AX
PrintNumber PROC NEAR
    push ax
    push bx
    push dx

    cmp ax, 10
    jb SingleDigit

    mov bx, 10
    xor dx, dx
    div bx

    push dx

    mov dl, al
    add dl, '0'
    call PrintChar

    pop dx
    add dl, '0'
    call PrintChar
    jmp NumberDone

SingleDigit:
    mov dl, al
    add dl, '0'
    call PrintChar

NumberDone:
    pop dx
    pop bx
    pop ax
    ret
PrintNumber ENDP

END main
```

- `OuterLoop` 和 `InnerLoop` 实现双重循环。
- `CALL PrintChar` 调用字符输出子程序。
- `CALL PrintNumber` 调用数字输出子程序。

- 两个子程序都使用 `RET` 返回。
- `push` 和 `pop` 用于保存、恢复寄存器，避免子程序破坏调用者需要的寄存器值。

这里使用了 DOS 的 `int 21H` 输出字符和字符串。

第三步：编译、链接并运行

先保存文件，然后在 DOSBox 中执行以下命令。

1. 编译

```
代码块
1   masm multable.asm;
```

编译成功生成 `MUL_TABLE.OBJ`。

```
D:\>masm multable.asm
Microsoft (R) Macro Assembler Version 5.00
Copyright (C) Microsoft Corp 1981-1985, 1987.  All rights reserved.

Object filename [multable.OBJ]:
Source listing  [NUL.LST]:
Cross-reference [NUL.CRF]:

  51688 + 464856 Bytes symbol space free

       0 Warning Errors
       0 Severe  Errors
```

2. 链接

```
代码块
1   link multable.obj;
```

链接成功生成 `MUL_TABLE.EXE`。

```
D:\>link multable.obj

Microsoft (R) Overlay Linker  Version 3.60
Copyright (C) Microsoft Corp 1983-1987.  All rights reserved.

Run File [MULTABLE.EXE]:
List File [NUL.MAP]:
Libraries [.LIB]:
```

3. 运行

代码块

```
multable
```

```
D:\>multable
The 9 multiplication table:
9*1=9  9*2=18  9*3=27  9*4=36  9*5=45  9*6=54  9*7=63  9*8=72  9*9=81
8*1=8  8*2=16  8*3=24  8*4=32  8*5=40  8*6=48  8*7=56  8*8=64
7*1=7  7*2=14  7*3=21  7*4=28  7*5=35  7*6=42  7*7=49
6*1=6  6*2=12  6*3=18  6*4=24  6*5=30  6*6=36
5*1=5  5*2=10  5*3=15  5*4=20  5*5=25
4*1=4  4*2=8  4*3=12  4*4=16
3*1=3  3*2=6  3*3=9
2*1=2  2*2=4
1*1=1
```

第四步：理解过程调用中的寄存器保存

以 `PrintChar` 为例：

代码块

```
PrintChar PROC NEAR
    push ax
    push dx

    mov ah, 02H
    int 21H

    pop dx
    pop ax
    ret
PrintChar ENDP
```

执行 `CALL PrintChar` 时，CPU 会把返回地址压入堆栈，然后跳转到子程序。

子程序内部：

1. `push ax`：保存原来的 AX。
2. `push dx`：保存原来的 DX。
3. `int 21H`：调用 DOS 输出字符。
4. `pop dx`、`pop ax`：按相反顺序恢复寄存器。
5. `ret`：从堆栈中取出返回地址，回到 `CALL` 后面的指令。

注意：`PUSH` 和 `POP` 必须保持后进先出的顺序。 如果保存顺序是 AX、DX，恢复顺序就应当是 DX、AX。

第五步：编写 C 语言程序

在 `D:\myassembly` 目录下新建文件 `multable.c`：

代码块

```c
#include <stdio.h>

void print_item(int i, int j)
{
    printf("%d*%d=%d  ", i, j, i * j);
}

int main(void)
{
    int i, j;

    printf("The 9 multiplication table:\n");

    for (i = 9; i >= 1; i--)
    {
        for (j = 1; j <= i; j++)
        {
            print_item(i, j);
        }

        printf("\n");
    }

    return 0;
}
```

- `print_item()`：负责输出一个乘法表达式，例如 `9*3=27`。

- `main()`：主函数，负责控制两个循环。
- `i` 从 9 递减到 1，控制每一行。
- `j` 从 1 递增到 `i`，控制每一行输出多少个乘法表达式。
- `print_item(i, j)` 是一次函数调用，对应汇编中可以看到的 `call` 指令。
- 函数执行完毕后返回调用位置，对应汇编中的 `ret` 指令。

这里特意把乘法表达式单独写成函数，而不是直接放在循环中，这样生成的汇编代码更容易观察过程调用。

第六步：使用 GCC 编译并运行

编译：

```
代码块
1   gcc multable.c -o multable.exe
```

运行：

```
代码块
1   multable.exe
```

```
D:\myassembly>gcc multable.c -o multable.exe

D:\myassembly>multable.exe
The 9 multiplication table:
9*1=9  9*2=18  9*3=27  9*4=36  9*5=45  9*6=54  9*7=63  9*8=72  9*9=81
8*1=8  8*2=16  8*3=24  8*4=32  8*5=40  8*6=48  8*7=56  8*8=64
7*1=7  7*2=14  7*3=21  7*4=28  7*5=35  7*6=42  7*7=49
6*1=6  6*2=12  6*3=18  6*4=24  6*5=30  6*6=36
5*1=5  5*2=10  5*3=15  5*4=20  5*5=25
4*1=4  4*2=8  4*3=12  4*4=16
3*1=3  3*2=6  3*3=9
2*1=2  2*2=4
1*1=1
```

第七步：生成 GCC 汇编代码

在 cmd 窗口中输入：

```
代码块
```

```
gcc -S -masm=intel -O0 multable.c -o multable_gcc.s
```

参数	作用
-S	只生成汇编代码，不进行链接
-masm=intel	使用 Intel 汇编语法
-O0	关闭优化，让汇编代码更容易理解
-o mul_table_gcc.s	指定生成文件的名称

`打开 mul_table_gcc.s`，理解一下指令含义：

代码块

```
call    print_item
```

这条指令对应 C 语言中的：

代码块

```
print_item(i, j);
```

另外：

代码块

```
call    printf
```

对应 C 语言中的 `printf()` 函数调用。

注意，Windows x64 环境下 GCC 生成的汇编代码会使用 `rcx`、`rdx` 等寄存器传递参数，和前面 DOSBox 中的 16 位 MASM 程序有所不同，这是正常的。

multable_gcc.s

```
        .file   "multable.c"
        .intel_syntax noprefix
        .text
        .section .rdata,"dr"
.LC0:
        .ascii "%d*%d=%d  \0"
        .text
        .globl  "print_item"
        .def    "print_item";   .scl    2;  .type   32; .endef
        .seh_proc   "print_item"
"print_item":
        push    rbp
        .seh_pushreg    rbp
        mov rbp, rsp
        .seh_setframe   rbp, 0
        sub rsp, 32
        .seh_stackalloc 32
        .seh_endprologue
        mov DWORD PTR 16[rbp], ecx
        mov DWORD PTR 24[rbp], edx
        mov eax, DWORD PTR 16[rbp]
        imul    eax, DWORD PTR 24[rbp]
        mov r8d, eax
        mov edx, DWORD PTR 24[rbp]
        mov eax, DWORD PTR 16[rbp]
        lea rcx, .LC0[rip]
        mov r9d, r8d
        mov r8d, edx
        mov edx, eax
        call    "printf"
        nop
        add rsp, 32
        pop rbp
        ret
        .seh_endproc
        .section .rdata,"dr"
.LC1:
        .ascii "The 9 multiplication table:\0"
        .text
        .globl  "main"
        .def    "main"; .scl    2;  .type   32; .endef
        .seh_proc   "main"
"main":
        push    rbp
```

第八步：给汇编代码添加中文注释

multable_gcc.s*

```
.file   "multable.c"                # 源文件名为 multable.c
.intel_syntax noprefix              # 使用 Intel 汇编语法
.text                               # 进入代码段

.section .rdata,"dr"                # 定义只读数据段
.LC0:
    .ascii "%d*%d=%d  \0"           # printf 的格式字符串：输出 i*j=结果，末尾有两个空格和结束符

.text                               # 切换到代码段
.globl "print_item"                 # 声明 print_item 为全局符号
.def "print_item"; .scl 2; .type 32; .endef  # 定义函数符号信息
.seh_proc "print_item"              # 开始记录函数的异常处理信息

"print_item":
    push rbp                        # 保存调用者的栈帧指针
    .seh_pushreg rbp                # 记录保存 rbp 的操作
    mov rbp, rsp                    # 建立当前函数的栈帧
    .seh_setframe rbp, 0            # 记录栈帧基址
    sub rsp, 32                     # 分配 32 字节局部栈空间
    .seh_stackalloc 32              # 记录栈空间分配信息
    .seh_endprologue                # 函数序言结束

    mov DWORD PTR 16[rbp], ecx      # 将第一个参数 i 保存到栈中
    mov DWORD PTR 24[rbp], edx      # 将第二个参数 j 保存到栈中

    mov eax, DWORD PTR 16[rbp]      # 取出 i，存入 eax
    imul eax, DWORD PTR 24[rbp]     # eax = i * j，计算乘积
    mov r8d, eax                    # 暂存乘积，作为 printf 的第三个数据参数

    mov edx, DWORD PTR 24[rbp]      # 取出 j，存入 edx
    mov eax, DWORD PTR 16[rbp]      # 取出 i，存入 eax
    lea rcx, .LC0[rip]              # 将格式字符串地址放入 rcx，作为 printf 的第一个参数

    mov r9d, r8d                    # 将乘积放入 r9d，作为 printf 的第四个参数
    mov r8d, edx                    # 将 j 放入 r8d，作为 printf 的第三个参数
    mov edx, eax                    # 将 i 放入 edx，作为 printf 的第二个参数
    call "printf"                   # 调用 printf，输出 i*j=乘积

    nop                             # 空操作指令，不改变程序状态
    add rsp, 32                     # 释放局部栈空间
    pop rbp                         # 恢复调用者的栈帧指针
    ret                             # 返回调用位置
.seh_endproc                        # print_item 的异常处理信息结束
```